

Project Description

Attached is a frontend project consisting of five buttons (or tabs). Each button corresponds to one of five database tables. When a button is selected, the corresponding table is displayed, showing 10 randomly selected rows.

In addition, there are two additional buttons that allow the user to:

- Add a random row to one of the tables.
- Remove a random row from one of the tables.

The frontend is already configured to communicate with a backend file named `main.py`. If preferred, you may implement the backend using JavaScript; however, appropriate modifications to the frontend will be required to ensure compatibility.

The Dockerfile in the github repository with the notes should cover your dependencies for this project. Alternatively, you can install the dependencies specified in notes 12 to complete the project.

Requirements

You are required to implement a backend in Python consisting of two modules: `main.py` and `crud.py`.

`main.py`

The `main.py` file should:

- Serve as the entry point of the backend application.
- Establish and expose API endpoints to the frontend.
- Handle incoming requests for:
 - Retrieving table data (limited to 10 randomly selected rows per request).
 - Triggering insertion of random rows.
 - Triggering deletion of random rows.
- Delegate all database-related operations to functions defined in `crud.py`.

`crud.py`

The `crud.py` module should:

- Manage all direct interactions with the database designed in Project 2.
- Implement functions for:
 - Querying tables and returning random subsets of rows.
 - Inserting randomly generated rows into tables.
 - Deleting randomly selected rows from tables.
- Serve as the data access layer separating database logic from application logic.
- You can extract columns like this:

```
colnames=[desc[0] for desc in cur.description]
```

- Hint: in order for your data to be properly displayed, after extracting columns and rows from table, combine them like this and return:

```
return [dict(zip(colnames, row)) for row in rows]
```

Frontend Table Naming Requirement

The frontend component (`App.jsx`) contains a constant array that defines the table names used to interact with the backend:

```
const TABLES = [...]
```

You are required to modify this array so that the table names exactly match the table names defined in your backend (`main.py`) and your database schema from Project 2.

Failure to ensure consistency between:

- `App.jsx` table names
- Backend table validation in `main.py`
- Actual database table names

will result in failed API requests and incorrect or missing data display.

Important Note

Ensure that all table identifiers are consistent across the entire stack:

- Frontend (`App.jsx`)
- Backend (`main.py`)
- Database schema (Project 2)

Additional Requirements

You must also reuse the database migration script developed in Project 2 to ensure proper schema creation and data migration for integration with the frontend system.